

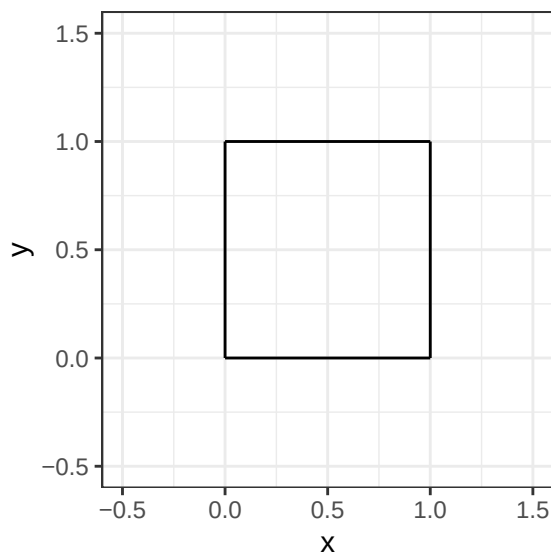
Lab Three – Matrices and Data Frames in R

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with “**Solution:**”.
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.

Consider the unit square depicted in Figure 1.

```
1 ggplot() +  
2   geom_segment(aes(x = 0, xend = 0,  
3                     y=0, yend = 1))+  
4   geom_segment(aes(x = 0, xend = 1,  
5                     y=1, yend = 1))+  
6   geom_segment(aes(x = 1, xend = 1,  
7                     y=0, yend = 1))+  
8   geom_segment(aes(x = 0, xend = 1,  
9                     y=0, yend = 0)) +  
10  theme_bw() +  
11  xlim(-0.5, 1.5)+  
12  ylim(-0.5, 1.5)+  
13  labs(x = "x",  
14        y = "y")
```

Figure 1: The unit square.



The unit square can be defined by two basis vectors

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

From these vectors, we can compute the corner vertices.

$p_0 = (0, 0) = (x_0, y_0)$	(The origin)
$p_1 = v_1 = (x_1, y_1)$	(The first basis vector)
$p_2 = v_1 + v_2 = (x_2, y_2)$	(The sum of basis vectors)
$p_3 = v_1 = (x_3, y_3)$	(The second basis vector)

Note that the segments that form the boundary go from p_0 to p_1 to p_2 to p_3 .

Consider the matrix M to be the matrix we get from binding the two basis vectors at the column,

$$\mathbf{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

1 Question 1

Let's transform $\mathbf{M}$ to demonstrate an interesting linear algebra property.

1.1 Part a

Create the matrix $\mathbf{A}$ in R.

$$\mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

Solution:

```
1 (A = matrix(c(1,5,-7,9),nrow=2,ncol=2))
```

```
      [,1] [,2]
[1,]     1  -7
[2,]     5   9
```

1.2 Part b

Compute the product below in R and store it in an object $\mathbf{T}$.

$$\mathbf{T} = \mathbf{A}\mathbf{M}.$$

Solution:

```
1 M = matrix(c(1,0,0,1),nrow=2,ncol=2)
2 T = A %*% M
3 (T)
```

```
      [,1] [,2]
[1,]     1  -7
[2,]     5   9
```

1.3 Part c

Create `basis.vector.1` (first column of $\mathbf{T}$) and `basis.vector.2` (second column of $\mathbf{T}$) in R.

Solution:

```
1 (basis.vector.1 = T[,1])
```

```
[1] 1 5
```

```
1 (basis.vector.2 = T[,2])
```

```
[1] -7 9
```

1.4 Part d

Compute the points p_0 , p_1 , p_2 , and p_3 .

Solution:

```
1 (p0 = c(0,0))
```

```

[1] 0 0
1 (p1 = basis.vector.1)

[1] 1 5
1 (p2 = basis.vector.1 + basis.vector.2)

[1] -6 14
1 (p3 = basis.vector.2)

[1] -7 9

```

1.5 Part e

Copy and paste the code for the unit square. Edit the code to draw the segments that form the boundary based on the points in Part d.

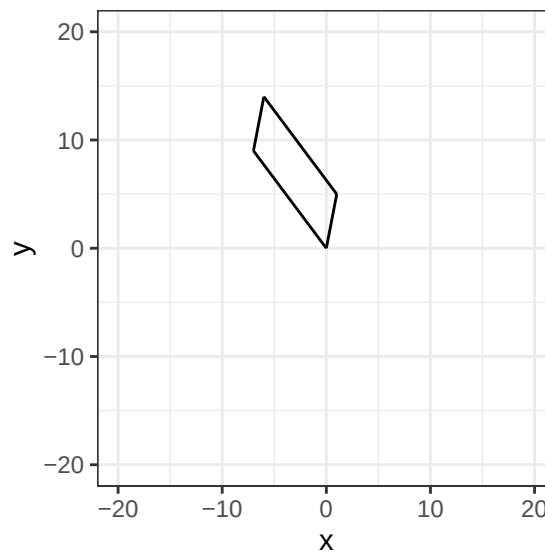
Solution:

```

1 ggplot() +
2   geom_segment(aes(x = p0[1], xend = p1[1],
3                     y=p0[2], yend = p1[2]))+
4   geom_segment(aes(x = p1[1], xend = p2[1],
5                     y=p1[2], yend = p2[2]))+
6   geom_segment(aes(x = p2[1], xend = p3[1],
7                     y=p2[2], yend = p3[2]))+
8   geom_segment(aes(x = p3[1], xend = p0[1],
9                     y=p3[2], yend = p0[2])) +
10  theme_bw() +
11  xlim(-20, 20)+
12  ylim(-20, 20)+
13  labs(x = "x",
14       y = "y")

```

Figure 2: My new parallelogram.



1.6 Part f

Interestingly, the determinant of $\mathbf{A}$ gives us the area of the shape plotted in Part e. Find the determinant of $\mathbf{A}$.

Solution:

```

1 (det(A))

[1] 44

```

2 Question 2

Recomplete Question 1 using a new matrix for **A**. This matrix is not as well behaved.

2.1 Part a

Create the matrix **A** in R.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Solution:

```
1 (A = matrix(c(1,2,2,4),nrow=2,ncol=2))
```

```
      [,1] [,2]  
[1,]     1     2  
[2,]     2     4
```

2.2 Part b

Compute the product below in R and store it in an object **T**.

$$\mathbf{T} = \mathbf{A}\mathbf{M}.$$

Solution:

```
1 M = matrix(c(1,0,0,1),nrow=2,ncol=2)  
2 (T = A %*% M)
```

```
      [,1] [,2]  
[1,]     1     2  
[2,]     2     4
```

2.3 Part c

Create **basis.vector.1** (first column of **T**) and **basis.vector.2** (second column of **T**) in R.

Solution:

```
1 (basis.vector.1 = T[,1])
```

```
[1] 1 2
```

```
1 (basis.vector.2 = 2 * basis.vector.1) #uh oh not full rank
```

```
[1] 2 4
```

Me when my matrix is not full rank ☹ (I'm never gonna get a true inverse)

2.4 Part d

Compute the points p_0 , p_1 , p_2 , and p_3 .

Solution:

```
1 (p0 = c(0,0))
```

```
[1] 0 0
```

```
1 (p1 = basis.vector.1)
```

```
[1] 1 2
```

```
1 (p2 = basis.vector.1 + basis.vector.2)
```

```
[1] 3 6
```

```
1 (p3 = basis.vector.2)
```

```
[1] 2 4
```

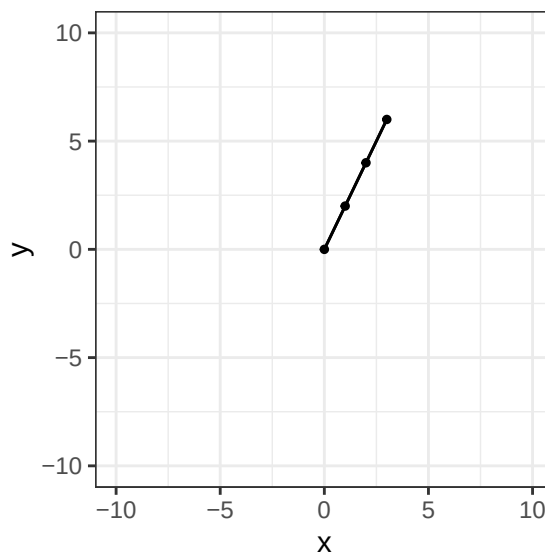
2.5 Part e

Copy and paste the code for plotting the unit square. Edit the code to draw the segments that form the boundary based on the points in Part d.

Solution:

```
1 ggplot() +
2   geom_segment(aes(x = p0[1], xend = p1[1],
3                     y=p0[2], yend = p1[2]))+
4   geom_point(aes(x = p0[1], y = p0[2]), size = 1) +
5   geom_segment(aes(x = p1[1], xend = p2[1],
6                     y=p1[2], yend = p2[2]))+
7   geom_point(aes(x = p1[1], y = p1[2]), size = 1) +
8   geom_segment(aes(x = p2[1], xend = p3[1],
9                     y=p2[2], yend = p3[2]))+
10  geom_point(aes(x = p2[1], y = p2[2]), size = 1) +
11  geom_segment(aes(x = p3[1], xend = p0[1],
12                    y=p3[2], yend = p0[2])) +
13  geom_point(aes(x = p3[1], y = p3[2]), size = 1) +
14  theme_bw() +
15  xlim(-10, 10)+
16  ylim(-10, 10)+
17  labs(x = "x",
18        y = "y")
```

Figure 3: My sad parallelogram.



2.6 Part f

Interestingly, the determinant of $\mathbf{A}$ gives us the area of the shape plotted in Part e. Find the determinant of $\mathbf{A}$.

Solution:

```
1 (det(A))
```

```
[1] 0
```

3 Question 3

We can conduct a shear transformation that keeps the area of the shape the same but horizontally or vertically slants the object.

In fact, this is one of the ways artists can make something look like it's leaning or being pushed. Other applications where this is useful include fluid dynamics and crystallography, where the area of a deformed object must remain constant, or in photography where perspective in photos can be adjusted.

A shear transformation is implemented by altering the basis vectors we start with. For vertical slanting,

$$\mathbf{M}_v = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

and for horizontal slanting

$$\mathbf{M}_h = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}.$$

In both cases, $k \in \mathbb{R}$ is called a shear factor and its sign determines the direction of the slant and its magnitude determines the severity of the slant.

3.1 Part a

Use the following to generate a new shape. Compare the shape and its area to your answer in Question 1.

$$\mathbf{M}_v = \begin{bmatrix} 1 & k=0.5 \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

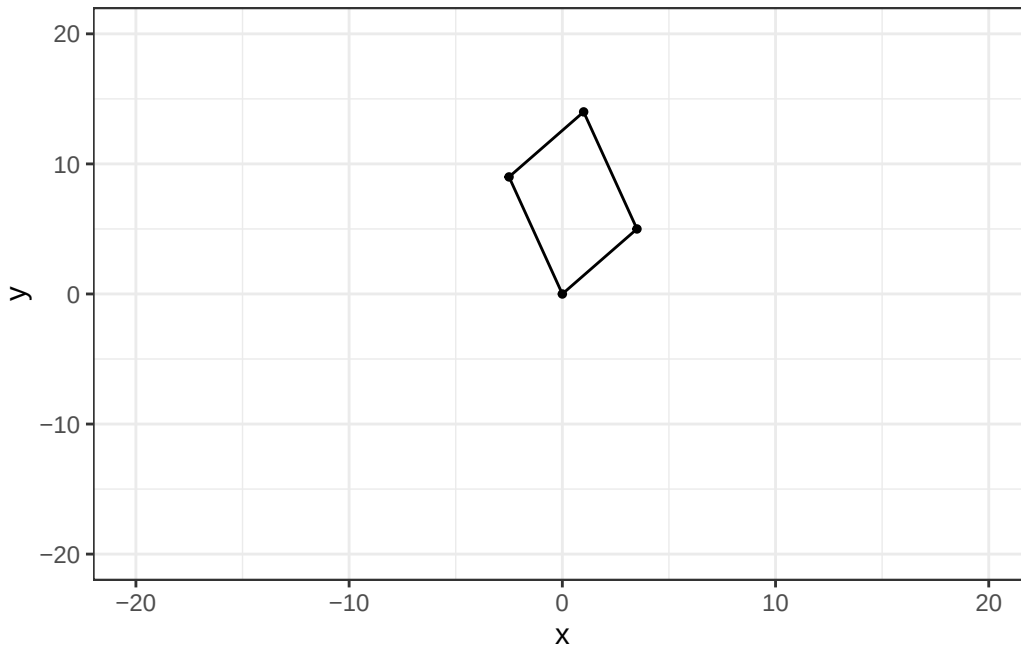
Solution:

```
1 #Let's plot!
2 #| label: fig-square-v4
3 #| fig-cap: "Vertically Sheared Parallelogram"
4 #| fig-width: 3
5 #| fig-height: 3
6 #| scale: "80%"
7 #| fig-align: "center"
8 #| size: "scriptsize"
9
10 #initializing my matrices
11 Mk = matrix(c(1,0,.5,1),nrow=2,ncol=2)
12 A = matrix(c(1,5,-7,9),nrow=2,ncol=2)
13 T = Mk %*% A
14
15 #it's basis vector time
16 basis.vector.1 = T[,1]
17 basis.vector.2 = T[,2]
18
19 #Grabbing my points
20 p0 = c(0,0)
21 p1 = basis.vector.1
22 p2 = basis.vector.1 + basis.vector.2
23 p3 = basis.vector.2
24
25
26 ggplot() +
27   geom_segment(aes(x = p0[1], xend = p1[1],
28                   y=p0[2], yend = p1[2]))+
29   geom_point(aes(x = p0[1], y = p0[2]), size = 1) +
30   geom_segment(aes(x = p1[1], xend = p2[1],
```

```

31         y=p1[2], yend = p2[2]))+
32 geom_point(aes(x = p1[1], y = p1[2]), size = 1) +
33 geom_segment(aes(x = p2[1], xend = p3[1],
34                 y=p2[2], yend = p3[2]))+
35 geom_point(aes(x = p2[1], y = p2[2]), size = 1) +
36 geom_segment(aes(x = p3[1], xend = p0[1],
37                 y=p3[2], yend = p0[2])) +
38 geom_point(aes(x = p3[1], y = p3[2]), size = 1) +
39 theme_bw() +
40 xlim(-20, 20)+
41 ylim(-20, 20)+
42 labs(x = "x",
43       y = "y")

```



The area of the shape (a parallelogram) is the magnitude of the determinant of the matrix

```
1 (det(T))
```

```
[1] 44
```

As expected, since the shear transformation M_k has determinant 1 (just like I_2) and because our determinant function is a homomorphism, $\det(AB) = \det A * \det B$. Thus the determinant is the same as in question 1.

3.2 Part b

Use the following to generate a new shape. Compare the shape and its area to your answer in Questions 1 and the vertical shear transform in part (a).

$$\mathbf{M}_h = \begin{bmatrix} 1 & 0 \\ k=0.5 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{A} = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

Solution:

```

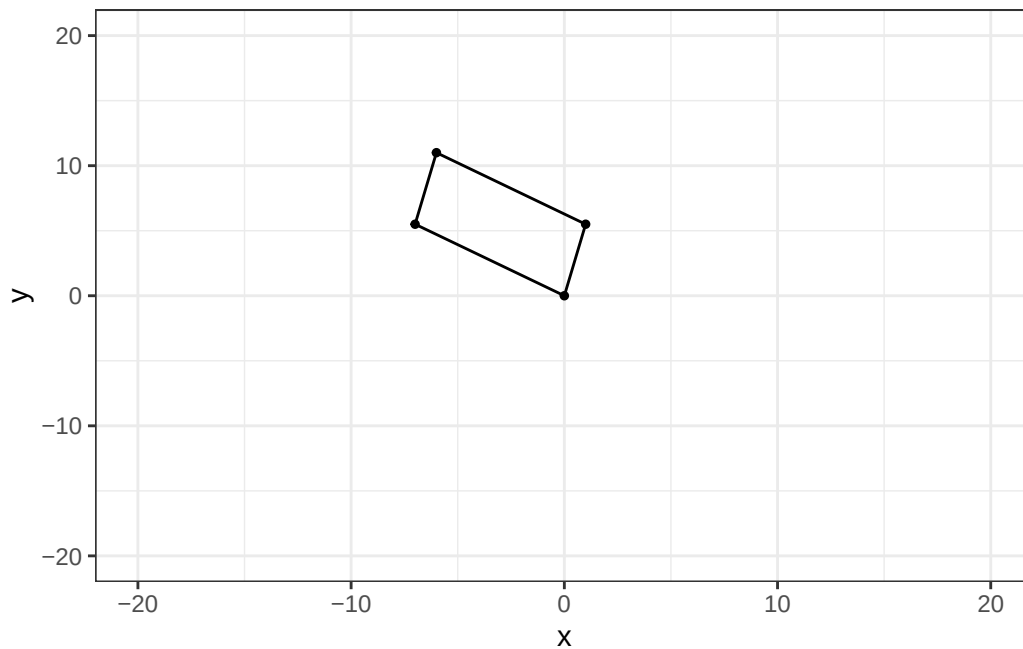
1 #Let's plot!
2 #| label: fig-square-v5
3 #| fig-cap: "Horizontally Sheered Parallelogram."
4 #| fig-width: 3

```

```

5  #| fig-height: 3
6  #| scale: "80%"
7  #| fig-align: "center"
8  #| size: "scriptsize"
9
10 #initializing my matrices
11 Mk = matrix(c(1,.5,0,1),nrow=2,ncol=2)
12 A = matrix(c(1,5,-7,9),nrow=2,ncol=2)
13 T = Mk %*% A
14
15 #it's basis vector time
16 basis.vector.1 = T[,1]
17 basis.vector.2 = T[,2]
18
19 #Grabbing my points
20 p0 = c(0,0)
21 p1 = basis.vector.1
22 p2 = basis.vector.1 + basis.vector.2
23 p3 = basis.vector.2
24
25
26 ggplot() +
27   geom_segment(aes(x = p0[1], xend = p1[1],
28                   y=p0[2], yend = p1[2]))+
29   geom_point(aes(x = p0[1], y = p0[2]), size = 1) +
30   geom_segment(aes(x = p1[1], xend = p2[1],
31                   y=p1[2], yend = p2[2]))+
32   geom_point(aes(x = p1[1], y = p1[2]), size = 1) +
33   geom_segment(aes(x = p2[1], xend = p3[1],
34                   y=p2[2], yend = p3[2]))+
35   geom_point(aes(x = p2[1], y = p2[2]), size = 1) +
36   geom_segment(aes(x = p3[1], xend = p0[1],
37                   y=p3[2], yend = p0[2])) +
38   geom_point(aes(x = p3[1], y = p3[2]), size = 1) +
39   theme_bw() +
40   xlim(-20, 20)+
41   ylim(-20, 20)+
42   labs(x = "x",
43        y = "y")

```



Once again, since the determinant of our matrix M_k is 1, we expect the determinant to be the same as in question 1. Let's check!

```
1 det(T)
```

```
[1] 44
```

3.3 Part c

Can we shear vertically and horizontally at the same time? Use the following to generate a new shape. Compare the shape and its area to your answers in parts (a) and (b).

$$M_{vh} = \begin{bmatrix} 1 & k = 0.5 \\ k = 0.5 & 1 \end{bmatrix} \quad \text{and} \quad A = \begin{bmatrix} 1 & -7 \\ 5 & 9 \end{bmatrix}$$

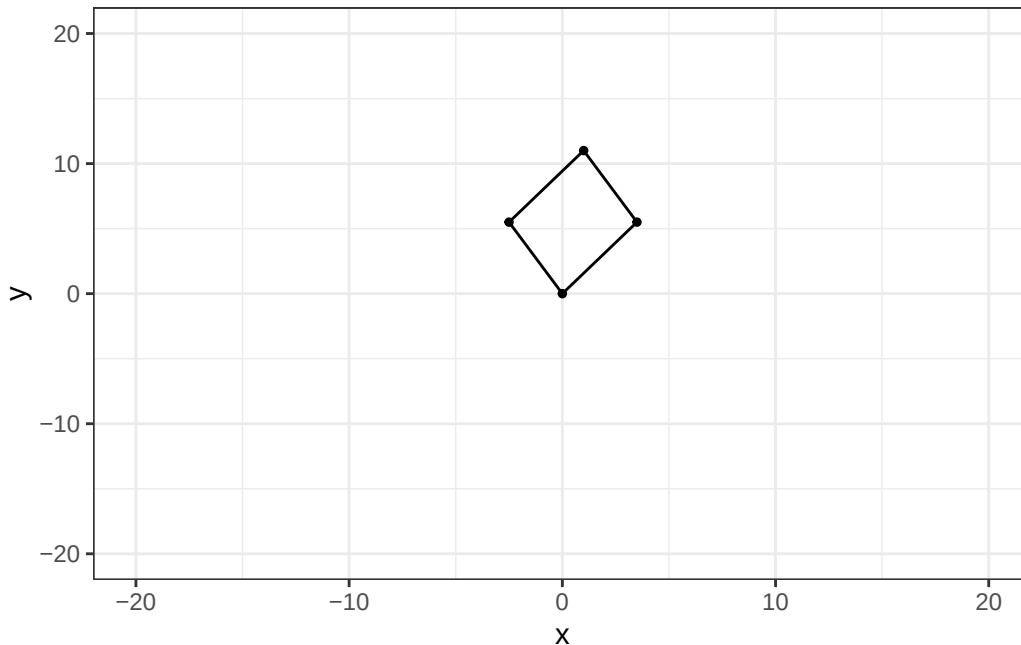
Solution:

```
1 #Let's plot!
2 #| label: fig-square-v6
3 #| fig-cap: "Doubly sheered Parallelogram"
4 #| fig-width: 3
5 #| fig-height: 3
6 #| scale: "80%"
7 #| fig-align: "center"
8 #| size: "scriptsize"
9
10
11 #initializing my matrices
12 Mk = matrix(c(1,.5,.5,1),nrow=2,ncol=2)
13 A = matrix(c(1,5,-7,9),nrow=2,ncol=2)
14 T = Mk %*% A
15
16 #it's basis vector time
17 basis.vector.1 = T[,1]
18 basis.vector.2 = T[,2]
19
20 #Grabbing my points
```

```

21 p0 = c(0,0)
22 p1 = basis.vector.1
23 p2 = basis.vector.1 + basis.vector.2
24 p3 = basis.vector.2
25
26
27 ggplot() +
28   geom_segment(aes(x = p0[1], xend = p1[1],
29                     y=p0[2], yend = p1[2]))+
30   geom_point(aes(x = p0[1], y = p0[2]), size = 1) +
31   geom_segment(aes(x = p1[1], xend = p2[1],
32                     y=p1[2], yend = p2[2]))+
33   geom_point(aes(x = p1[1], y = p1[2]), size = 1) +
34   geom_segment(aes(x = p2[1], xend = p3[1],
35                     y=p2[2], yend = p3[2]))+
36   geom_point(aes(x = p2[1], y = p2[2]), size = 1) +
37   geom_segment(aes(x = p3[1], xend = p0[1],
38                     y=p3[2], yend = p0[2])) +
39   geom_point(aes(x = p3[1], y = p3[2]), size = 1) +
40   theme_bw() +
41   xlim(-20, 20)+
42   ylim(-20, 20)+
43   labs(x = "x",
44         y = "y")

```



This matrix no longer has determinant one (because the product of the off-diagonal is no longer 0). So this parallelogram should have a different area from the sheer transformations and the original. Let's check!

```

1 det(T)

```

```

[1] 33

```

4 Question 4

4.1 Part a

Create a data frame `ladder` with the following data. Note your data frame should have three columns; I split it to save vertical space here.

x	y	group	x	y	group
0	0	1	1	8	1
0	20	1	0	10	1
1	0	2	1	10	2
1	20	2	0	12	2
0	2	3	1	12	3
1	2	3	0	14	3
0	4	4	1	14	4
1	4	4	0	16	4
0	6	5	1	16	5
1	6	5	0	18	5
0	8	5	1	18	5

Solution:

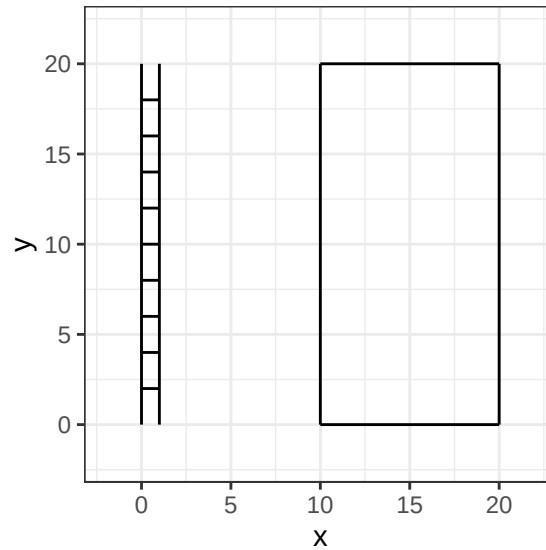
```
1 x = c(0,0,1,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0,1)
2 y = c(0,20,0,20,2,2,4,4,6,6,8,8,10,10,12,12,14,14,16,16,18,18)
3 group = c(1,1,2,2,3,3,4,4,5,5,6,6,7,7,8,8,9,9,10,10,11,11)
4 ladder = data.frame(x=x,y=y,group=group)
```

4.2 Part b

Remove `#|eval: false` in the code below. The result should look like a ladder and a very rectangular house if part a is correct.

```
1 ggplot() +
2   geom_path(data = ladder,
3             aes(x = x, y = y, group = group)) +
4     geom_segment(aes(x = 10, xend = 10,
5                       y=0, yend = 20))+
6     geom_segment(aes(x = 10, xend = 20,
7                       y=20, yend = 20))+
8     geom_segment(aes(x = 20, xend = 20,
9                       y=0, yend = 20))+
10    geom_segment(aes(x = 10, xend = 20,
11                     y=0, yend = 0)) +
12    theme_bw() +
13    xlim(-2,22) +
14    ylim(-2,22)
```

Figure 4: A ladder.



4.3 Part c

Use `as.matrix()` to create a 22×2 matrix **A** containing the `x` and `y` observations from the `ladder` data frame. Use matrix multiplication to compute

$$\mathbf{T} = \mathbf{A}\mathbf{M}_h.$$

Recall,

$$\mathbf{M}_h = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$$

and use $k = 0.5$.

solution

```
1 M = matrix(c(1,.5,0,1),nrow=2,ncol=2)
2 ladderNoGroups = ladder[,c("x","y")]
3 A = as.matrix(ladderNoGroups,nrow=22,ncol=2)
4
5 T = A %*% M
6 (T)
```

```
      [,1] [,2]
[1,]    0    0
[2,]   10   20
[3,]    1    0
[4,]   11   20
[5,]    1    2
[6,]    2    2
[7,]    2    4
[8,]    3    4
[9,]    3    6
[10,]   4    6
[11,]   4    8
[12,]   5    8
[13,]   5   10
[14,]   6   10
[15,]   6   12
[16,]   7   12
```

```
[17,]    7   14
[18,]    8   14
[19,]    8   16
[20,]    9   16
[21,]    9   18
[22,]   10   18
```

4.4 Part d

Create a data frame `leaning.ladder` with the `x` and `y` equal to the first and second column of `T`, respectively, and the same values of `group` from `ladder`.

Solution:

```
1 leaning.ladder = data.frame(x=T[,1],y=T[,2],group=ladder[,c("group")])
```

4.5 Part e

Remove `eval: false` in the code below. The result should look like a ladder and a very rectangular house if part a is correct. The new leaning ladder should look like it is leaning on the house.

```
1 ggplot() +
2   geom_path(data = ladder,
3             aes(x = x, y = y, group = group, color = "Starting Ladder")) +
4   geom_path(data = leaning.ladder,
5             aes(x = x, y = y, group = group, color = "Leaning Ladder")) +
6   geom_segment(aes(x = 10, xend = 10,
7                   y=0, yend = 20))+
8   geom_segment(aes(x = 10, xend = 20,
9                   y=20, yend = 20))+
10  geom_segment(aes(x = 20, xend = 20,
11                  y=0, yend = 20))+
12  geom_segment(aes(x = 10, xend = 20,
13                  y=0, yend = 0)) +
14  theme_bw() +
15  xlim(-2,22) +
16  ylim(-2,22) +
17  scale_color_manual("", values=c("darkred", "grey"))
```

Figure 5: A ladder and a leaning ladder.

